Machine Coding: Web Audio Player

Problem: Build a web audio player with a playlist: play / pause, next / previous, seek bar, time display, volume and mute, shuffle, repeat, loading and error states.

Tech stack: React (Vite, JavaScript), plain CSS (default, not specified)

Time limit: 60 minutes (default, not specified)

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List the interactions:** every action and its expected result. This is your test checklist.
3. **Design the architecture:** components / modules, one responsibility each.
4. **Design the state:** what is stored, what is derived, who owns it.
5. **Design the data and contracts:** example data, component props, API shapes.
6. **Trace one flow end to end:** input -> state -> side effect -> output.
7. **Find the hard parts early:** solve them on paper before typing.
8. **Build in layers:** a working version first, then one working layer at a time.
9. **Cover states and edge cases:** loading, empty, error, boundaries.
10. **Wrap up:** trade-offs, what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements, questions, assumptions	5 min
Architecture, state, contracts	8 min
Core: `<audio>` + play / pause + time display	10 min
Seek bar + volume	10 min
Playlist, next / prev, ended	10 min
Shuffle, repeat, loading / error states	10 min

Phase	Time
Keyboard, polish, wrap-up	7 min

Think out loud

Say every decision and its reason as you make it: "*The `<audio>` element owns playback time, so I will read it from events instead of keeping my own timer.*" The interviewer grades your reasoning as much as your code. If you get stuck, say what you are trying and what you would check next.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Play and pause the current track.
- Show the title and artist of the current track.
- Show current time and total duration (`1:15 / 6:12`).
- Seek by clicking or dragging a progress bar.
- Next / previous track. "Previous" restarts the track if it has played more than 3 seconds.
- Volume slider and mute toggle.
- Playlist: click a track to play it; highlight the active one.
- Auto-advance to the next track when one ends.
- Repeat modes: off -> all -> one.
- Shuffle: random order without repeats, current track keeps playing.
- Loading indicator while buffering; clear message when a track fails.
- Keyboard: Space = play / pause, Left / Right = seek -5s / +5s.

Non-functional requirements

Area	What it means here
Performance	`timeupdate` fires ~4x per second: only the small time state updates; no polling timers.
Scalability	Playlist uses stable ids; the queue is an array of indexes, so a large playlist only costs one array.
Error handling	Media errors (404, decode, unsupported) show a message and stop; autoplay-blocked `play()` does not crash.

Area	What it means here
Accessibility	Native `<button>` and `<input type="range">`, `aria-label`, `aria-pressed`, `aria-current`, `aria-live` status, visible focus ring.
Reusability	All logic lives in a `useAudioPlayer(tracks)` hook; UI components are dumb and prop-driven.
Extensibility	New features (playback speed, Media Session, visualizer) attach to the hook without touching UI components.

Clarifying questions

1. Where do tracks come from: a static list, or an API? Do I need pagination?
2. Should the player autoplay on load? (Browsers block autoplay with sound, so I assume no.)
3. At the end of the playlist with repeat off: stop on the last track, or jump back to the first?
4. When a track fails: show an error and stop, or skip automatically?
5. Do we need a waveform / visualizer (Web Audio API), or is `<audio>` enough?
6. Should volume / last position persist across reloads?

Assumptions

- A static list of tracks with `{ id, title, artist, src }`; at least one track (empty list shows an empty state).
- No autoplay: playback starts on a user click.
- End of playlist with repeat off: stop on the last track; Next is disabled there.
- A failed track shows an error and stops; the user can pick another track (no auto-skip, so an all-broken playlist cannot loop forever).
- Plain `HTMLAudioElement`; Web Audio API visualizer is a stretch goal.
- No persistence.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	First track selected, paused, metadata loads, duration shown, seek enabled.
Click Play	Audio plays, button turns to Pause, playlist shows ♪ on the active row.
Click Pause	Audio pauses at the same position.
Drag the seek thumb	Thumb and time label follow the pointer; audio does not jump until release.
Release the seek thumb / click the bar	Audio jumps to that time.
Click Next	Next track in the queue loads; keeps playing if it was playing.

Action / input	Expected behavior / output
Click Next on the last track (repeat off)	Disabled.
Click Previous after 3s of playback	Current track restarts from 0.
Click Previous in the first 3s	Previous track loads (or restarts if it is the first track).
Track ends	Next track plays; with repeat all it wraps; with repeat one it loops; at the end with repeat off it stops.
Click a playlist row	That track loads and plays.
Click the active playlist row	Toggles play / pause.
Click Shuffle	Queue reshuffles, current track keeps playing, next tracks come in random order with no repeats.
Click Repeat	Cycles off -> all -> one.
Move volume slider	Volume changes; moving it to 0 shows muted icon.
Click Mute	Mutes / unmutes; unmuting at volume 0 restores 50%.
Press Space (nothing focused)	Toggles play / pause, page does not scroll.
Press Left / Right	Seeks -5s / +5s.
Track is buffering	"Loading..." status.
Track URL is broken	"Track not found or format not supported.", paused, seek disabled.
Browser blocks play()	Button goes back to Play, no crash.

3. Design the architecture

Module	Responsibility
`App`	Page shell; empty state when there are no tracks.
`AudioPlayer`	Wires the hook to the UI; owns the hidden `<audio>` and keyboard shortcuts.
`useAudioPlayer`	All player state and actions; syncs React state with the `<audio>` element.
`NowPlaying`	Title, artist, loading / error status.
`ProgressBar`	Seek slider + time labels; local drag state.
`Controls`	Shuffle, previous, play / pause, next, repeat buttons.
`VolumeControl`	Mute button + volume slider.

Module	Responsibility
`Playlist`	List of tracks, active highlight, click to play.
`utils/queue.js`	Pure functions: build (shuffled) queue, next / previous position.
`utils/time.js`	`formatTime(seconds)`.

```
App
└── AudioPlayer  ── useAudioPlayer(tracks) ──> <audio> (hidden, source of truth for time)
    ├── NowPlaying
    ├── ProgressBar     (local dragTime)
    ├── Controls
    ├── VolumeControl
    └── Playlist
```

Start with one component holding `<audio>` and a play button. Pull logic into the hook once there are more than a few handlers; split UI pieces once each has its own props.

4. Design the state / data model

State	Purpose
`queue`	Play order as track indexes, e.g. `[0, 1, 2, 3]` or shuffled `[2, 0, 3, 1]`.
`pos`	Position in the queue of the current track.
`isPlaying`	User intent: should audio be playing. The element follows it.
`status`	`'idle' | 'loading' | 'ready' | 'error'`.
`error`	Human-readable message for the current track's failure.
`currentTime`	Mirror of `audio.currentTime`, updated from `timeupdate`.
`duration`	Mirror of `audio.duration`, updated from `loadedmetadata` / `durationchange`.
`volume`, `muted`	Volume 0..1 and mute flag.
`repeat`	`'off' | 'all' | 'one'`.
`shuffle`	Whether the queue is shuffled.
`dragTime` (ProgressBar)	Seek position while the user drags; `null` when not dragging.

```
const [queue, setQueue] = useState(() => buildQueue(tracks.length, false, 0))
const [pos, setPos] = useState(0)
const [isPlaying, setIsPlaying] = useState(false)
```

```
const [status, setStatus] = useState('idle')
const [error, setError] = useState(null)
const [currentTime, setCurrentTime] = useState(0)
const [duration, setDuration] = useState(0)
const [volume, setVolume] = useState(0.8)
const [muted, setMuted] = useState(false)
const [repeat, setRepeat] = useState('off')
const [shuffle, setShuffle] = useState(false)
```

Senior note: derive, don't store

- `currentIndex = queue[pos]` and `track = tracks[currentIndex]`: derived, never stored, so they cannot go out of sync.
- `canNext = getNextPos(...) !== -1`: derived.
- Progress percent = `currentTime / duration`: derived in `ProgressBar`.
- **Ownership:** the `<audio>` element owns the real time, duration and buffering; React state only mirrors it from events. React owns the *intent* (`isPlaying`, `pos`, `volume`), and effects push intent into the element. `dragTime` is owned locally by `ProgressBar`, because nobody else cares about an uncommitted drag.

5. Design the data structures and contracts

Example data

```
export const TRACKS = [
  { id: 't1', title: 'SoundHelix Song 1', artist: 'SoundHelix',
    src: 'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3' },
  { id: 't2', title: 'SoundHelix Song 2', artist: 'SoundHelix',
    src: 'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-2.mp3' },
  // ...
  { id: 't4', title: 'Broken Link', artist: 'Demo (error state)',
    src: 'https://www.soundhelix.com/examples/mp3/does-not-exist.mp3' },
]
```

Why stable ids: they are React keys for the playlist rows, and the play effect depends on `track.id` so it re-runs exactly when the track changes. Indexes would break if the list were reordered. The broken track exists on purpose to demo the error state.

Contracts

If the list came from an API:

```
GET /api/playlists/:id
200 { "id": "p1", "tracks": [ { "id": "t1", "title": "...", "artist": "...", "src": "https://...",
"duration": 372 } ] }
```

Hook contract:

```
const player = useAudioPlayer(tracks)
// state:   track, currentIndex, isPlaying, status, error, currentTime, duration,
//          volume, muted, repeat, shuffle, canNext
// actions: togglePlay(), next(), prev(), seek(sec), seekBy(delta), playTrack(index),
//          toggleShuffle(), cycleRepeat(), changeVolume(0..1), toggleMute()
// element: audioProps -> spread onto <audio {...audioProps} />
```

Pure helpers:

```
buildQueue(length, shuffle, firstIndex) // -> number[] (current track first when shuffled)
getNextPos(pos, length, repeat)         // -> next pos, or -1 = stop
getPrevPos(pos, length, repeat)         // -> previous pos (wraps only with repeat 'all')
formatTime(seconds)                     // 75 -> "1:15", NaN/Infinity -> "0:00"
```

6. Trace the data flow

Click Next while playing:

```
User clicks ⏭
  ↓
next(): getNextPos(pos, queue.length, repeat) -> 2
  ↓
setPos(2)  → re-render: track = tracks[queue[2]], <audio src> changes
  ↓
Browser: loadstart → onLoadStart: status 'loading', currentTime 0, duration 0
  ↓
Effect [isPlaying, track.id] runs (after the new src is in the DOM)
  → audio.play() (promise; AbortError ignored if the user skips again quickly)
  ↓
loadedmetadata → duration = 425, status 'ready'
playing        → status 'ready'
timeupdate ×4/s → currentTime updates → ProgressBar + time label re-render
```

```
  ↓
UI: new title, 0:00 / 7:05, ♪ on row 3, Pause button
```

7. Important technical challenges

Challenge 1: Keeping React state and the `<audio>` element in sync

Problem: The audio element is imperative and has its own state (paused, time, src). If you call `audio.play()` inside the click handler for Next, it runs *before* React has rendered the new `src`, so it plays the old track. And `play()` returns a promise: clicking Next twice fast makes the first `play()` reject with `AbortError`; a browser autoplay block rejects with `NotAllowedError`. Unhandled, both show as uncaught errors, and the button would say "Pause" while nothing plays.

Solution: Store *intent* (`isPlaying`) in React. One effect makes the element follow it, keyed on `[isPlaying, track.id]`, so it runs after the commit, when the new `src` is already in the DOM. Ignore `AbortError`; for any other rejection set the intent back to false. Read time / duration back from media events.

```
useEffect(() => {
  const audio = audioRef.current
  if (!isPlaying) { audio.pause(); return }
  audio.play().catch((err) => {
    if (err.name === 'AbortError') return // src changed or pause() came first
    setIsPlaying(false)                   // autoplay blocked / unsupported
  })
}, [isPlaying, track.id])
```

Event handlers are passed as React props (`onTimeUpdate`, `onEnded`, ...) on `<audio>`, so they always see fresh state. With `addEventListener` inside a `useEffect(..., [])`, `onEnded` would capture the first `pos` forever (stale closure).

Challenge 2: Seek bar fights `timeupdate` while dragging

Problem: The slider value is `currentTime`. While the user drags the thumb to 2:00, `timeupdate` keeps setting `currentTime` to 0:03, 0:04... so the thumb jumps back under the finger. If you seek on every `onChange` instead, the browser fires dozens of seeks per drag, each one re-buffering.

Solution: Local `dragTime` state. While dragging, show `dragTime` and ignore the audio time; seek once on release (pointer up, key up, or blur).

```
const [dragTime, setDragTime] = useState(null)
const shown = dragTime ?? currentTime
```

```
const commit = () => {
  if (dragTime === null) return
  onSeek(dragTime)
  setDragTime(null)
}
<input type="range" value={shown}
  onChange={(e) => setDragTime(Number(e.target.value))}
  onPointerUp={commit} onKeyUp={commit} onBlur={commit} />
```

Challenge 3: Next / previous with shuffle and repeat

Problem: Picking a random track on every Next repeats songs and makes Previous impossible (where did we come from?). Toggling shuffle must not stop the current song. Repeat-one, repeat-all and the end of the list all need different "next" rules.

Solution: Keep a **queue** (array of track indexes) plus a **position**. Shuffle = Fisher-Yates over the other tracks, with the current track placed first, so it keeps playing and nothing repeats until the queue ends. Next / previous are pure functions of `(pos, length, repeat)`, easy to test. Repeat-one uses the element's native `loop` attribute, so `ended` never fires and no code is needed.

```
export function buildQueue(length, shuffle, firstIndex) {
  const order = Array.from({ length }, (_, i) => i)
  if (!shuffle) return order
  const rest = order.filter((i) => i !== firstIndex)
  for (let i = rest.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1))
    ;[rest[i], rest[j]] = [rest[j], rest[i]]
  }
  return [firstIndex, ...rest]
}

export function getNextPos(pos, length, repeat) {
  if (pos + 1 < length) return pos + 1
  return repeat === 'all' ? 0 : -1 // -1 = stop
}
```

Edge case: in a 1-track playlist with repeat all, "next" returns the same position, so `src` does not change and nothing reloads. `goTo()` handles it by restarting the track instead.

Challenge 4: Loading and error states from media events

Problem: A track can be slow (buffering mid-song), or broken (404, bad codec). `play()` alone does not tell you either; the UI would show a frozen Pause button.

Solution: Map media events to a small status machine and translate `MediaError.code` into a message.

Event	Status
`loadstart`	`loading` (also reset time / duration / error)
`loadedmetadata` , `playing`	`ready`
`waiting`	`loading` (buffering mid-play)
`error`	`error` + message, stop playing

```
onError: (e) => {
  setStatus('error')
  setError(MEDIA_ERRORS[e.currentTarget.error?.code] ?? 'Could not play this track.')
  setIsPlaying(false)
}
```

8. Implement in the correct order

#	Step	Working result
1	Vite app, `TRACKS` data, `App` with empty state	Page renders
2	`<audio src>` + play / pause button driven by `isPlaying` effect	Can play one track
3	`onTimeUpdate` / `onLoadedMetadata` + `formatTime`	`0:12 / 6:12` shown
4	Seek slider with `dragTime`	Click / drag to seek
5	Volume slider + mute	Volume works
6	Move logic into `useAudioPlayer` ; split components	Same app, cleaner
7	Queue + `pos` , `next` / `prev` , playlist click	Multiple tracks
8	`onEnded` auto-advance, repeat modes (`loop` for one)	Continuous play
9	Shuffle with `buildQueue`	Random order, no repeats
10	Status machine: loading / error, `play()` rejection	Robust states
11	Keyboard shortcuts, ARIA, focus styles	Accessible
12	Self-check pure helpers, polish CSS	Done

9. Cover all states and edge cases

State / edge case	Handling
Initial	First track, paused, status `idle` -> `loading` -> `ready` once metadata arrives.
Loading / buffering	`waiting` -> "Loading...", `aria-live` announces it.
Playing / paused	Button label + icon, ♪ on active row.
Error (404 / codec)	Message, paused, seek disabled (duration unknown).
Empty playlist	`App` shows "No tracks in the playlist." (hook never runs).
Duration unknown / live stream (`NaN` / `Infinity`)	`formatTime` -> `0:00`, seek disabled, `seek()` is a no-op.
Autoplay blocked	`play()` rejects `NotAllowedError` -> back to paused.
Fast double Next	First `play()` rejects `AbortError` -> ignored.
Seek drag	Thumb stays under the pointer; one seek on release.
Seek out of range (+5s near end)	Clamped to `[0, duration]`.
Last track, repeat off	Next disabled; ended -> stop.
Last track, repeat all	Wraps to first.
Repeat one	Native `loop`; Next button still skips.
1-track playlist / Previous on first track	Same position -> restart instead of a no-op.
Previous after 3s	Restart current track.
Shuffle toggled mid-song	Current track stays, keeps playing.
Click active row	Toggle play / pause instead of restarting.
Volume to 0	Treated as muted; unmute restores 50%.
Space while a button is focused	Global shortcut skipped (the button's own click handles it, no double toggle).
Track switch mid-load	`loadstart` resets time, duration and error from the previous track.

10. Final structure

```
_55_web_audio_player/
├── index.html
├── package.json
```

```
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── data/
    │   └── tracks.js
    ├── hooks/
    │   └── useAudioPlayer.js
    ├── utils/
    │   ├── queue.js
    │   └── time.js
    └── components/
        ├── AudioPlayer.jsx
        ├── NowPlaying.jsx
        ├── ProgressBar.jsx
        ├── Controls.jsx
        ├── VolumeControl.jsx
        └── Playlist.jsx
```

11. Full working code

Create the project with `npm create vite@latest _55_web_audio_player -- --template react`, then replace the files below.

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react-swc'

export default defineConfig({
  plugins: [react()],
  cacheDir: '.vite-cache',
})
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
```

```
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.jsx

```
import AudioPlayer from './components/AudioPlayer'
import { TRACKS } from './data/tracks'

export default function App() {
  return (
    <main className="app">
      <h1>Web Audio Player</h1>
      {TRACKS.length > 0 ? (
        <AudioPlayer tracks={TRACKS} />
      ) : (
        <p className="empty">No tracks in the playlist.</p>
      )}
    </main>
  )
}
```

src/data/tracks.js

```
// Stable ids: used as React keys and survive reordering / shuffling.
export const TRACKS = [
  {
    id: 't1',
    title: 'SoundHelix Song 1',
    artist: 'SoundHelix',
    src: 'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3',
  },
  {
    id: 't2',
    title: 'SoundHelix Song 2',
    artist: 'SoundHelix',
```

```
    src: 'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-2.mp3',
  },
  {
    id: 't3',
    title: 'SoundHelix Song 3',
    artist: 'SoundHelix',
    src: 'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-3.mp3',
  },
  {
    // Broken on purpose: demonstrates the error state.
    id: 't4',
    title: 'Broken Link',
    artist: 'Demo (error state)',
    src: 'https://www.soundhelix.com/examples/mp3/does-not-exist.mp3',
  },
]
```

src/utils/time.js

```
// 75 -> "1:15", 3725 -> "1:02:05". NaN / Infinity (unknown or live duration) -> "0:00".
export function formatTime(seconds) {
  if (!Number.isFinite(seconds) || seconds < 0) return '0:00'
  const total = Math.floor(seconds)
  const h = Math.floor(total / 3600)
  const m = Math.floor((total % 3600) / 60)
  const s = String(total % 60).padStart(2, '0')
  return h > 0 ? `${h}:${String(m).padStart(2, '0')}:${s}` : `${m}:${s}`
}
```

src/utils/queue.js

```
export const REPEAT_MODES = ['off', 'all', 'one']

// Play order as track indexes. Shuffled queues keep the current track first,
// so toggling shuffle never interrupts what is playing.
export function buildQueue(length, shuffle, firstIndex) {
  const order = Array.from({ length }, (_, i) => i)
  if (!shuffle) return order
  const rest = order.filter((i) => i !== firstIndex)
  for (let i = rest.length - 1; i > 0; i--) {
```

```
    const j = Math.floor(Math.random() * (i + 1)) // Fisher-Yates
    ;[rest[i], rest[j]] = [rest[j], rest[i]]
  }
  return [firstIndex, ...rest]
}

// Next queue position, or -1 when playback should stop (end of queue, no repeat).
export function getNextPos(pos, length, repeat) {
  if (pos + 1 < length) return pos + 1
  return repeat === 'all' ? 0 : -1
}

export function getPrevPos(pos, length, repeat) {
  if (pos > 0) return pos - 1
  return repeat === 'all' ? length - 1 : 0
}
```

src/hooks/useAudioPlayer.js

```
import { useCallback, useEffect, useRef, useState } from 'react'
import { buildQueue, getNextPos, getPrevPos, REPEAT_MODES } from '../utils/queue'

const MEDIA_ERRORS = {
  1: 'Playback was aborted.',
  2: 'Network error while loading the track.',
  3: 'The track could not be decoded.',
  4: 'Track not found or format not supported.',
}
const RESTART_THRESHOLD = 3 // seconds: "previous" restarts the track after this point

export function useAudioPlayer(tracks) {
  const audioRef = useRef(null)
  const [queue, setQueue] = useState(() => buildQueue(tracks.length, false, 0))
  const [pos, setPos] = useState(0)
  const [isPlaying, setIsPlaying] = useState(false) // user intent: the effect below makes the element
follow it
  const [status, setStatus] = useState('idle') // idle | loading | ready | error
  const [error, setError] = useState(null)
  const [currentTime, setCurrentTime] = useState(0)
  const [duration, setDuration] = useState(0)
  const [volume, setVolume] = useState(0.8)
```

```
  const [muted, setMuted] = useState(false)
  const [repeat, setRepeat] = useState('off')
  const [shuffle, setShuffle] = useState(false)

  const currentIndex = queue[pos]
  const track = tracks[currentIndex]
  const canNext = getNextPos(pos, queue.length, repeat) !== -1

  // Runs after React has put the new src on <audio>, so play() targets the new track.
  useEffect(() => {
    const audio = audioRef.current
    if (!isPlaying) {
      audio.pause()
      return
    }
    audio.play().catch((err) => {
      if (err.name === 'AbortError') return // src changed or pause() came first: harmless
      setIsPlaying(false) // NotAllowedError (autoplay policy) or unsupported source
    })
  }, [isPlaying, track.id])

  useEffect(() => {
    audioRef.current.volume = volume
    audioRef.current.muted = muted
  }, [volume, muted])

  const seek = useCallback((time) => {
    const audio = audioRef.current
    if (!Number.isFinite(audio.duration)) return // metadata not loaded or live stream
    audio.currentTime = Math.min(Math.max(time, 0), audio.duration)
    setCurrentTime(audio.currentTime)
  }, [])

  const seekBy = useCallback((delta) => seek(audioRef.current.currentTime + delta), [seek])
  const togglePlay = useCallback(() => setIsPlaying((p) => !p), [])

  // Same position again (1-track queue, "previous" on the first track): restart, src does not change.
  const goTo = (newPos) => {
    if (newPos !== pos) return setPos(newPos)
    seek(0)
    if (isPlaying) audioRef.current.play().catch(() => {})
  }
```

```
const next = () => {
  const n = getNextPos(pos, queue.length, repeat)
  if (n === -1) setIsPlaying(false)
  else goTo(n)
}

const prev = () => {
  if (audioRef.current.currentTime > RESTART_THRESHOLD) return seek(0)
  goTo(getPrevPos(pos, queue.length, repeat))
}

const playTrack = (index) => {
  if (index === currentIndex) return togglePlay()
  setPos(queue.indexOf(index))
  setIsPlaying(true)
}

const toggleShuffle = () => {
  const newQueue = buildQueue(tracks.length, !shuffle, currentIndex)
  setShuffle(!shuffle)
  setQueue(newQueue)
  setPos(newQueue.indexOf(currentIndex))
}

const cycleRepeat = () =>
  setRepeat((r) => REPEAT_MODES[(REPEAT_MODES.indexOf(r) + 1) % REPEAT_MODES.length])

const changeVolume = (v) => {
  setVolume(v)
  setMuted(v === 0)
}

const toggleMute = () => {
  if (muted && volume === 0) setVolume(0.5) // unmuting at 0 would still be silent
  setMuted((m) => !m)
}

// Spread onto <audio>: React keeps these handlers fresh, so no stale closures.
const audioProps = {
  ref: audioRef,
  src: track.src,
```

```
    preload: 'metadata',
    loop: repeat === 'one', // native loop: "ended" never fires, no code needed
    onLoadStart: () => {
      setStatus('loading')
      setError(null)
      setCurrentTime(0)
      setDuration(0)
    },
    onLoadedMetadata: (e) => {
      setDuration(e.currentTarget.duration)
      setStatus('ready')
    },
    onDurationChange: (e) => setDuration(e.currentTarget.duration),
    onTimeUpdate: (e) => setCurrentTime(e.currentTarget.currentTime),
    onWaiting: () => setStatus('loading'),
    onPlaying: () => setStatus('ready'),
    onEnded: next,
    onError: (e) => {
      setStatus('error')
      setError(MEDIA_ERRORS[e.currentTarget.error?.code] ?? 'Could not play this track.')
      setIsPlaying(false)
    },
  }

  return {
    audioProps,
    track,
    currentIndex,
    isPlaying,
    status,
    error,
    currentTime,
    duration,
    volume,
    muted,
    repeat,
    shuffle,
    canNext,
    togglePlay,
    next,
    prev,
    seek,
```

```
    seekBy,
    playTrack,
    toggleShuffle,
    cycleRepeat,
    changeVolume,
    toggleMute,
  }
}
```

src/components/AudioPlayer.jsx

```
import { useEffect } from 'react'
import { useAudioPlayer } from '../hooks/useAudioPlayer'
import NowPlaying from './NowPlaying'
import ProgressBar from './ProgressBar'
import Controls from './Controls'
import VolumeControl from './VolumeControl'
import Playlist from './Playlist'

export default function AudioPlayer({ tracks }) {
  const player = useAudioPlayer(tracks)
  const { togglePlay, seekBy } = player

  // Global shortcuts. Skip when a control has focus: Space on a focused button already clicks it.
  useEffect(() => {
    const onKeyDown = (e) => {
      if (e.target.closest?.('input, button, textarea, select, [contenteditable]')) return
      if (e.code === 'Space') {
        e.preventDefault() // stop the page from scrolling
        togglePlay()
      } else if (e.code === 'ArrowRight') seekBy(5)
      else if (e.code === 'ArrowLeft') seekBy(-5)
    }
    window.addEventListener('keydown', onKeyDown)
    return () => window.removeEventListener('keydown', onKeyDown)
  }, [togglePlay, seekBy])

  return (
    <section className="player" aria-label="Audio player">
      <audio {...player.audioProps} />
      <NowPlaying track={player.track} status={player.status} error={player.error} />
```

```
      <ProgressBar
        currentTime={player.currentTime}
        duration={player.duration}
        onSeek={player.seek}
      />
      <Controls
        isPlaying={player.isPlaying}
        shuffle={player.shuffle}
        repeat={player.repeat}
        canNext={player.canNext}
        onToggle={player.togglePlay}
        onPrev={player.prev}
        onNext={player.next}
        onShuffle={player.toggleShuffle}
        onRepeat={player.cycleRepeat}
      />
      <VolumeControl
        volume={player.volume}
        muted={player.muted}
        onVolume={player.changeVolume}
        onToggleMute={player.toggleMute}
      />
      <Playlist
        tracks={tracks}
        currentIndex={player.currentIndex}
        isPlaying={player.isPlaying}
        onSelect={player.playTrack}
      />
    </section>
  )
}
```

src/components/NowPlaying.jsx

```
export default function NowPlaying({ track, status, error }) {
  return (
    <div className="now-playing">
      <h2>{track.title}</h2>
      <p className="artist">{track.artist}</p>
      <p className={`status status-${status}`} role="status" aria-live="polite">
        {status === 'loading' && 'Loading…'}
```

```
        {status === 'error' && error}
      </p>
    </div>
  )
}
```

src/components/ProgressBar.jsx

```
import { useState } from 'react'
import { formatTime } from '../utils/time'

export default function ProgressBar({ currentTime, duration, onSeek }) {
  // While dragging, show the drag position and ignore timeupdate, so the thumb does not jump back.
  const [dragTime, setDragTime] = useState(null)
  const canSeek = Number.isFinite(duration) && duration > 0
  const shown = dragTime ?? currentTime

  const commit = () => {
    if (dragTime === null) return
    onSeek(dragTime)
    setDragTime(null)
  }

  return (
    <div className="progress">
      <span className="time">{formatTime(shown)}</span>
      <input
        type="range"
        min={0}
        max={canSeek ? duration : 0}
        step="any"
        value={canSeek ? shown : 0}
        disabled={!canSeek}
        aria-label="Seek"
        aria-valuetext={`${formatTime(shown)} of ${formatTime(duration)}`}
        style={{ '--fill': `${canSeek ? (shown / duration) * 100 : 0}%` }}
        onChange={(e) => setDragTime(Number(e.target.value))}
        onPointerUp={commit}
        onKeyUp={commit}
        onBlur={commit}
      />
```

```
      <span className="time">{formatTime(duration)}</span>
    </div>
  )
}
```

src/components/Controls.jsx

```
export default function Controls({
  isPlaying,
  shuffle,
  repeat,
  canNext,
  onToggle,
  onPrev,
  onNext,
  onShuffle,
  onRepeat,
}) {
  return (
    <div className="controls">
      <button
        className={shuffle ? 'on' : ''}
        aria-label="Shuffle"
        aria-pressed={shuffle}
        onClick={onShuffle}
      >
        🔀
      </button>
      <button aria-label="Previous track" onClick={onPrev}>
        ⏮
      </button>
      <button className="play" aria-label={isPlaying ? 'Pause' : 'Play'} onClick={onToggle}>
        {isPlaying ? '⏸' : '▶'}
      </button>
      <button aria-label="Next track" onClick={onNext} disabled={!canNext}>
        ⏭
      </button>
      <button
        className={repeat !== 'off' ? 'on' : ''}
        aria-label={`Repeat: ${repeat}`}
        onClick={onRepeat}
```

```
      >
        {repeat === 'one' ? '🔂' : '🔁'}
      </button>
    </div>
  )
}
```

src/components/VolumeControl.jsx

```
export default function VolumeControl({ volume, muted, onVolume, onToggleMute }) {
  const level = muted ? 0 : volume
  const icon = level === 0 ? '🔇' : level < 0.5 ? '🔉' : '🔊'

  return (
    <div className="volume">
      <button aria-label={muted ? 'Unmute' : 'Mute'} onClick={onToggleMute}>
        {icon}
      </button>
      <input
        type="range"
        min={0}
        max={1}
        step={0.01}
        value={level}
        aria-label="Volume"
        style={{ '--fill': `${level * 100}%` }}
        onChange={(e) => onVolume(Number(e.target.value))}
      />
    </div>
  )
}
```

src/components/Playlist.jsx

```
export default function Playlist({ tracks, currentIndex, isPlaying, onSelect }) {
  return (
    <ol className="playlist" aria-label="Playlist">
      {tracks.map((t, i) => {
        const active = i === currentIndex
        return (
```

```
          <li key={t.id}>
            <button
              className={active ? 'active' : ''}
              aria-current={active ? 'true' : undefined}
              onClick={() => onSelect(i)}
            >
              <span className="index">{active && isPlaying ? '♪' : i + 1}</span>
              <span className="meta">
                <strong>{t.title}</strong>
                <small>{t.artist}</small>
              </span>
            </button>
          </li>
        )
      })}
    </ol>
  )
}
```

src/index.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #0f172a;
  color: #e2e8f0;
}

.app {
  max-width: 420px;
  margin: 0 auto;
  padding: 24px 16px;
}

h1 {
  font-size: 1.25rem;
  text-align: center;
```

```
}

.player {
  background: #1e293b;
  border-radius: 16px;
  padding: 20px;
  display: grid;
  gap: 16px;
}

.now-playing {
  text-align: center;
}

.now-playing h2 {
  margin: 0;
  font-size: 1.1rem;
}

.artist {
  margin: 4px 0 0;
  color: #94a3b8;
}

.status {
  min-height: 1.2em;
  margin: 6px 0 0;
  font-size: 0.85rem;
}

.status-error {
  color: #f87171;
}

.progress,
.volume {
  display: flex;
  align-items: center;
  gap: 8px;
}

.time {
```

```
  font-variant-numeric: tabular-nums;
  font-size: 0.8rem;
  color: #94a3b8;
  min-width: 3.2em;
  text-align: center;
}

input[type='range'] {
  flex: 1;
  height: 6px;
  appearance: none;
  border-radius: 3px;
  background: linear-gradient(to right, #38bdf8 var(--fill), #334155 var(--fill));
  cursor: pointer;
}

input[type='range']:disabled {
  cursor: not-allowed;
  opacity: 0.5;
}

input[type='range']::-webkit-slider-thumb {
  appearance: none;
  width: 14px;
  height: 14px;
  border-radius: 50%;
  background: #f8fafc;
}

.controls {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 12px;
}

button {
  font: inherit;
  color: inherit;
  background: none;
  border: 0;
  border-radius: 8px;
```

```
  padding: 6px 8px;
  cursor: pointer;
  font-size: 1.2rem;
}

button:hover:not(:disabled) {
  background: #334155;
}

button:disabled {
  opacity: 0.35;
  cursor: not-allowed;
}

button:focus-visible,
input:focus-visible {
  outline: 2px solid #38bdf8;
  outline-offset: 2px;
}

.controls .on {
  background: #0c4a6e;
}

.controls .play {
  width: 56px;
  height: 56px;
  border-radius: 50%;
  background: #38bdf8;
  color: #0f172a;
  font-size: 1.4rem;
}

.controls .play:hover {
  background: #7dd3fc;
}

.playlist {
  list-style: none;
  margin: 0;
  padding: 0;
  border-top: 1px solid #334155;
```

```
}

.playlist button {
  width: 100%;
  display: flex;
  align-items: center;
  gap: 12px;
  text-align: left;
  font-size: 0.95rem;
  padding: 10px 8px;
}

.playlist button.active {
  background: #0c4a6e;
}

.playlist .index {
  width: 1.5em;
  text-align: center;
  color: #94a3b8;
}

.playlist .meta {
  display: grid;
}

.playlist small {
  color: #94a3b8;
}

.empty {
  text-align: center;
  color: #94a3b8;
}
```

Self-check for the pure helpers (`node check.mjs`)

```
import assert from 'node:assert/strict'
import { formatTime } from './src/utils/time.js'
import { buildQueue, getNextPos, getPrevPos } from './src/utils/queue.js'
```

```
assert.equal(formatTime(0), '0:00')
assert.equal(formatTime(75.9), '1:15')
assert.equal(formatTime(3725), '1:02:05')
assert.equal(formatTime(NaN), '0:00')
assert.equal(formatTime(Infinity), '0:00')

assert.deepEqual(buildQueue(4, false, 2), [0, 1, 2, 3])
for (let k = 0; k < 50; k++) {
  const q = buildQueue(5, true, 3)
  assert.equal(q[0], 3)
  assert.deepEqual([...q].sort(), [0, 1, 2, 3, 4])
}

assert.equal(getNextPos(0, 3, 'off'), 1)
assert.equal(getNextPos(2, 3, 'off'), -1)
assert.equal(getNextPos(2, 3, 'one'), -1)
assert.equal(getNextPos(2, 3, 'all'), 0)
assert.equal(getPrevPos(1, 3, 'off'), 0)
assert.equal(getPrevPos(0, 3, 'off'), 0)
assert.equal(getPrevPos(0, 3, 'all'), 2)
console.log('self-check ok')
```

12. Wrap-up

Edge cases handled

- `play()` promise rejections (AbortError on fast skips, NotAllowedError on autoplay block).
- Seek drag does not fight `timeupdate`; one seek per drag; clamped to the duration.
- Unknown / infinite duration disables seeking.
- Media errors mapped to readable messages; error does not auto-loop.
- End of queue, wrap-around, repeat-one, 1-track playlist, Previous restart rule.
- Shuffle keeps the current song and never repeats within a cycle.
- Global keyboard shortcuts do not double-fire on focused controls.

Trade-offs to mention

- **`isPlaying` is intent, not a mirror of `audio.paused`.** Simple and predictable, but if the OS pauses audio (headphones unplugged, another tab takes over) the button can say "Pause" while paused. Fix: also listen to `pause` / `play` events, or use the Media Session API.
- **`timeupdate` (~4 Hz)** re-renders the player 4 times a second. Fine for this size; for a smoother bar use `requestAnimationFrame` while playing, or update the bar via a ref instead of state.

- **One `useState` per field** is readable; with more actions a `useReducer` would make transitions (e.g. `TRACK_CHANGED` resetting time + status together) explicit.
- **No auto-skip on error:** safer (no infinite loop over broken tracks), but less seamless.
- **`tracks` is assumed static:** the queue is built once. If the list changes, rebuild the queue in an effect keyed on the track ids.

What to add with more time

- Tests: unit tests for `queue.js` / `time.js` (already pure); React Testing Library for controls, mocking `HTMLMediaElement.prototype.play` / `pause`.
- Media Session API: lock-screen controls, artwork, hardware media keys.
- Persist volume, shuffle, repeat and last position in `localStorage`.
- Buffered ranges shown on the progress bar (`audio.buffered`).
- Playback speed, drag-to-reorder playlist, "play next" queue.
- Web Audio API visualizer (`AnalyserNode` + canvas) or equalizer.
- Preload the next track for gapless playback.
- Virtualize the playlist for thousands of tracks.

13. Counter questions the interviewer may ask

Requirements

Q1. Why not autoplay when the page loads?

Chrome, Safari and Firefox block audible autoplay until the user interacts with the page; `play()` rejects with `NotAllowedError`. Starting on a click is the reliable behavior. If autoplay is required, start muted and show an "unmute" button.

Q2. What if the product wants the player to keep playing while the user navigates the app?

Lift the player out of the routed page: render `<audio>` and the hook once in the app layout (or a context provider) above the router, and expose the actions through context. Routes only render controls; the element is never unmounted.

State design

Q3. Why store `isPlaying` instead of reading `audio.paused`?

`audio.paused` is not React state, so reading it does not re-render. Storing intent gives one source of truth for the UI, and the effect makes the element follow it. The cost is drift if something outside React pauses the audio (see trade-offs).

Q4. Why keep `queue` + `pos` instead of `currentIndex`?

With shuffle, "next" is not `currentIndex + 1`. A queue stores the play order once, so Next / Previous are trivial and shuffle never repeats. Without shuffle the queue is just `[0..n-1]` and `pos === currentIndex`.

Q5. Would you use `useReducer` here?

Yes, if it grows. Track changes reset several fields together (time, duration, status, error); a reducer action like `TRACK_LOADING` makes that atomic and testable without a DOM. For this size, `useState` keeps the code shorter.

Q6. Why is `dragTime` local to `ProgressBar` and not in the hook?

Nothing else needs an uncommitted drag. Keeping it local means dragging re-renders only the progress bar, and the hook API stays "seek to X" instead of "start / move / end drag".

Challenge 1: syncing with `<audio>`

Q7. Why not call `audio.play()` directly inside `next()`?

`next()` only schedules `setPos`; the `src` attribute is updated later, at commit. Calling `play()` in the handler would play the *old* track, then the src swap aborts it. The effect runs after commit, when the new src is in place.

Q8. Why `onTimeUpdate` as a JSX prop instead of `addEventListener`?

React re-binds JSX handlers every render, so `onEnded={next}` always uses the current `pos` and `repeat`. With `addEventListener` in an effect with `[]` deps, the handler captures the first render's state (stale closure), unless you re-subscribe on every change or use refs.

Q9. What is an `AbortError` from `play()` and why ignore it?

`play()` returns a promise that is rejected with `AbortError` when the load is interrupted, e.g. `src` changes or `pause()` is called before playback starts. It is expected when the user skips quickly, so it is not a real error.

Challenge 2: seeking

Q10. Why not seek on every `onChange` of the slider?

Each `currentTime` write starts a seek, which may fetch a new byte range from the server. Dozens per drag waste bandwidth and stutter. Seeking once on release is smooth and cheap; the label still updates live from `dragTime`.

Q11. What if the server does not support HTTP range requests?

The browser cannot jump to an unbuffered position; seeking only works within the downloaded part, or restarts. `audio.seekable` tells you which ranges are seekable; disable or clamp the slider to those ranges, and ask the backend to serve `Accept-Ranges: bytes`.

Challenge 3: queue / shuffle / repeat

Q12. Why Fisher-Yates instead of `arr.sort(() => Math.random() - 0.5)`?

The sort trick gives a biased, engine-dependent order. Fisher-Yates is O(n) and every permutation is equally likely.

Q13. When repeat-all wraps with shuffle on, should the order reshuffle?

Many players reshuffle each cycle. Here the order repeats; to reshuffle, call `buildQueue` again in `next()` when `getNextPos` wraps to 0, keeping the next track first to avoid playing the same song twice in a row.

Q14. Why use the native `loop` attribute for repeat-one?

The element restarts itself without firing `ended`, with no gap and no code. Handling `ended` + `currentTime = 0` + `play()` works too, but adds a small gap and one more code path.

Challenge 4: loading / errors

Q15. What if a track fails: why not skip to the next one automatically?

Auto-skip is nicer UX, but if every track is broken (offline, CDN down) it loops forever. If required, auto-skip with a counter: stop after N consecutive failures and show "Playback failed".

Q16. What if the network is slow?

`preload="metadata"` fetches only headers at first; buffering mid-song fires `waiting` and shows "Loading…". With more time: show `audio.buffered` on the bar and preload the next track near the end of the current one.

Implementation and performance

Q17. What if the playlist has 100,000 tracks?

Rendering is the bottleneck: virtualize the list (`react-window`), and load tracks page by page from the API. The queue is an array of numbers (cheap); shuffling 100k is O(n) and fast. Keep the playing track's metadata separate from the page that is loaded.

Q18. `timeupdate` re-renders the whole player 4x per second. Is that a problem?

At this size, no. If it grows, move `currentTime` into a small context or a separate store used only by `ProgressBar`, or update the bar's CSS variable through a ref inside `requestAnimationFrame` so React does not re-render at all.

Q19. How do you test this?

Pure helpers with plain unit tests (`buildQueue`, `getNextPos`, `formatTime`). Components with React Testing Library: mock `HTMLMediaElement.prototype.play` to return a resolved promise, fire `loadedmetadata`, `timeupdate`, `ended`, `error` with `fireEvent`, assert the UI. One E2E test with a short local audio file.

Trade-offs and "why not X?"

Q20. Why not the Web Audio API (`AudioContext`) for playback?

`<audio>` streams, seeks, buffers and handles codecs for free. Web Audio (`decodeAudioData`) needs the whole file in memory. Use Web Audio *on top* of the element (`createMediaElementSource` -> `AnalyserNode`) only for visualizers or EQ; note it requires CORS headers on the audio host, or the output is silent.

Q21. Why not just use `<audio controls>`?

It is accessible and free, but looks different per browser, cannot show a custom playlist, shuffle or repeat, and cannot be styled. A custom UI on top of a hidden element gives full control while keeping the browser's media engine.

Q22. How would you support lock-screen / hardware media keys?

Media Session API: set `navigator.mediaSession.metadata = new MediaMetadata({ title, artist, artwork })` on track change, and `setActionHandler('play' | 'pause' | 'nexttrack' | 'previoustrack' | 'seekto', ...)` to call the hook's actions.